Impact Compendium Database - Technical Specification

Document Version: 1.0
Date: October 21, 2025
Author: IT Architecture Team
Project: CGIAR/Alliance Bioversity & CIAT Impact Compendium

1. Project Overview

The **Impact Compendium Database** is a comprehensive web application designed to register, manage, and visualize research studies, indicators, and contextual data for the CGIAR/Alliance Bioversity & CIAT organization.

Purpose

- Centralize impact research data across CGIAR initiatives
- Enable authenticated researchers to create, search, and manage studies
- Provide comprehensive reporting and visualization capabilities
- Support three study types: Impact Studies, Outcome Studies, and Impact Outcome Stories

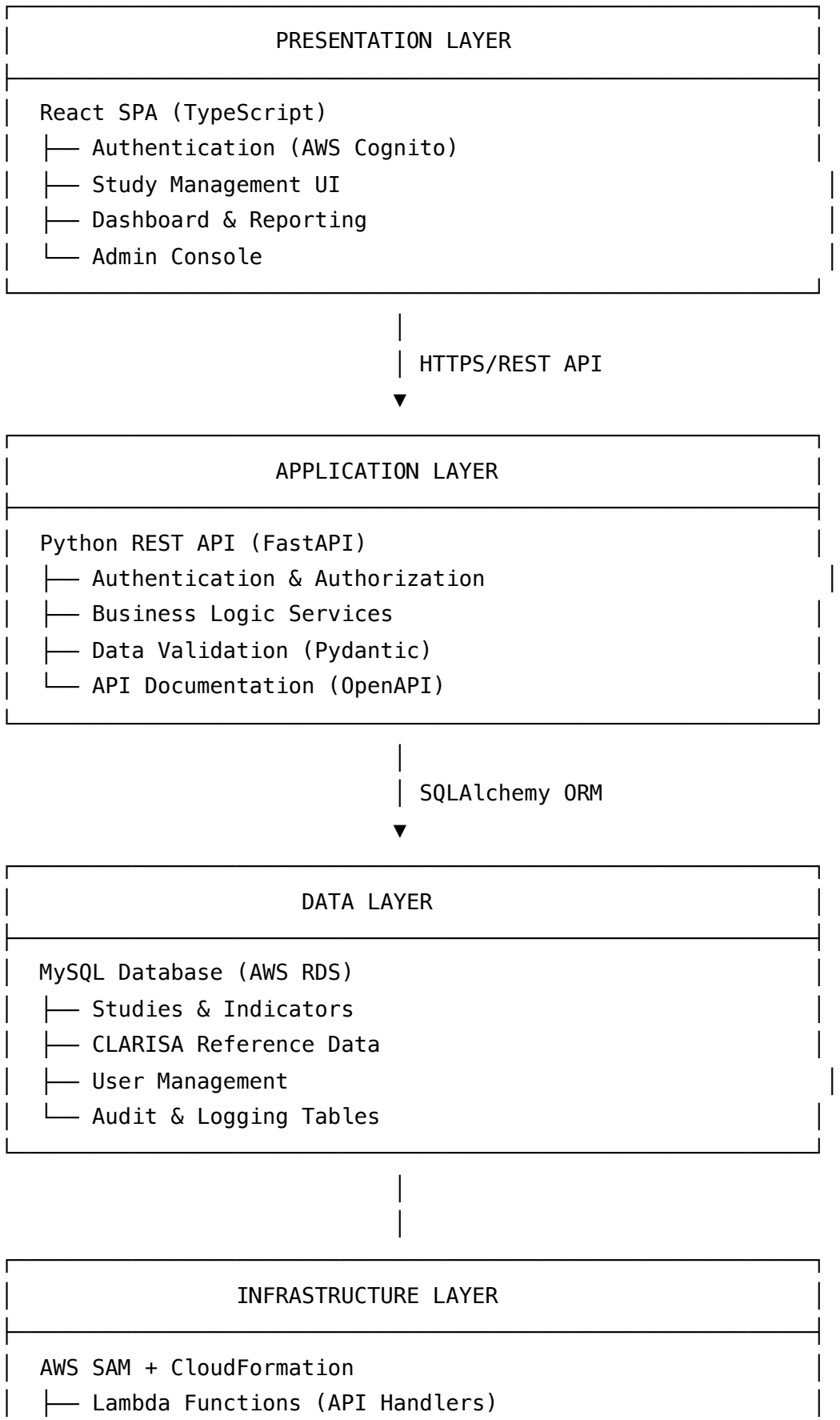
Key Stakeholders

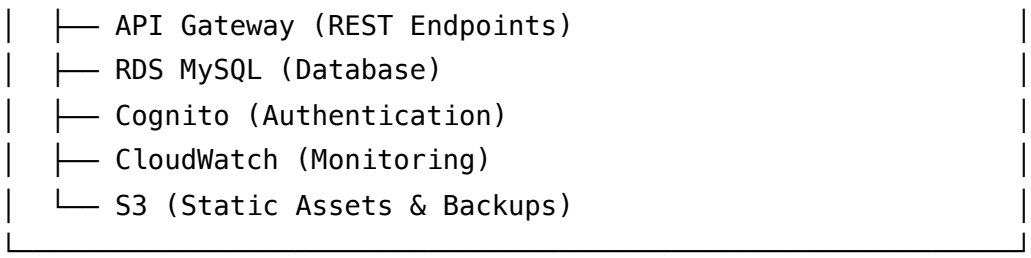
- **Researchers**: Create and manage studies, indicators, and metadata
- **Program Managers**: Monitor impact across initiatives and regions
- **Data Analysts**: Generate reports and export data for analysis
- **System Administrators**: Manage users, categories, and system configuration

Goals

- Streamline impact data collection and management
- Improve data quality through structured forms and validation
- Enable cross-initiative collaboration and knowledge sharing
- Support evidence-based decision making through comprehensive reporting

2. Architectural Overview





3. Functional Modules

3.1 Authentication Module

- **AWS Cognito Integration**: User registration, login, password reset
- **Role-based Access Control**: Admin, Researcher, Viewer roles
- **JWT Token Management**: Secure API access with token refresh
- **Multi-factor Authentication**: Optional 2FA for enhanced security

3.2 Study Management Module

- **CRUD Operations**: Create, read, update, delete studies
- **Multi-step Form Wizard**: Guided study creation process
- **Indicator Management**: Link studies to impact indicators
- **Relationship Mapping**: Context relationships and dependencies
- **Version Control**: Track study modifications and history

3.3 Search and Filter Module

- **Full-text Search**: Search across study titles, descriptions, and metadata
- **Advanced Filters**: Filter by initiative, region, indicator type, date range
- **Faceted Search**: Category-based filtering with result counts
- **Export Functionality**: CSV, Excel, and PDF export options

3.4 Dashboard and Reporting Module

- **Impact Visualization**: Charts and graphs for impact metrics
- **Aggregated Reports**: Cross-initiative and regional summaries
- **Custom Dashboards**: User-configurable dashboard widgets

- **Scheduled Reports**: Automated report generation and distribution

3.5 Admin Console Module

- **User Management**: Create, modify, and deactivate user accounts
- **Reference Data Management**: Manage initiatives, centers, countries
- **System Configuration**: Application settings and parameters
- **Audit Logging**: Track system changes and user activities

4. Backend Architecture (Python REST API)

4.1 Project Structure

```
app/
├── main.py                # FastAPI application entry point
├── config/
│   ├── __init__.py
│   ├── database.py        # Database connection configuration
│   └── settings.py        # Environment variables and settings
├── routers/
│   ├── __init__.py
│   ├── auth.py            # Authentication endpoints
│   ├── studies.py         # Study CRUD operations
│   ├── indicators.py      # Indicator management
│   ├── admin.py           # Admin console endpoints
│   └── reports.py         # Reporting and export endpoints
├── models/
│   ├── __init__.py
│   ├── study.py           # SQLAlchemy study models
│   ├── indicator.py       # Indicator models
│   ├── user.py            # User and authentication models
│   └── clarisa.py         # CLARISA reference data models
├── schemas/
│   ├── __init__.py
│   ├── study.py           # Pydantic study schemas
│   ├── indicator.py       # Indicator validation schemas
│   └── user.py            # User management schemas
├── services/
│   ├── __init__.py
│   ├── study_service.py   # Business logic for studies
│   ├── auth_service.py    # Authentication business logic
│   └── report_service.py  # Report generation logic
└── utils/
    ├── __init__.py
    ├── security.py        # JWT and password utilities
    └── validators.py      # Custom validation functions
```

4.2 Key API Endpoints

```
# Authentication
POST /auth/login           # User authentication
POST /auth/refresh         # Token refresh
POST /auth/logout          # User logout

# Studies
GET  /studies              # List studies with filters
POST /studies              # Create new study
GET  /studies/{id}         # Get study details
PUT  /studies/{id}         # Update study
DELETE /studies/{id}       # Delete study

# Indicators
GET  /indicators           # List available indicators
POST /studies/{id}/indicators  # Link indicators to study

# Admin
GET  /admin/users          # User management
POST /admin/initiatives    # Manage initiatives
GET  /admin/audit-logs     # System audit logs

# Reports
GET  /reports/dashboard    # Dashboard data
POST /reports/export       # Export filtered data
```

4.3 Data Validation & ORM

- **Pydantic Schemas**: Request/response validation and serialization
- **SQLAlchemy ORM**: Database abstraction and relationship management
- **Alembic Migrations**: Database schema version control
- **Custom Validators**: Business rule validation

5. Frontend Architecture (React)

5.1 Project Structure

```
src/
├── components/
│   ├── ui/                # shadcn/ui components
│   ├── forms/             # Form components and wizards
│   ├── tables/            # Data table components
│   └── charts/            # Visualization components
├── pages/
│   ├── Dashboard.tsx      # Main dashboard
│   ├── Studies.tsx        # Study management
│   ├── Reports.tsx        # Reporting interface
│   └── Admin.tsx          # Admin console
├── hooks/
│   ├── useAuth.ts         # Authentication hook
│   ├── useStudies.ts      # Study data management
│   └── useApi.ts          # API integration hook
├── context/
│   ├── AuthContext.tsx    # Authentication state
│   └── AppContext.tsx     # Global application state
├── services/
│   ├── api.ts             # Axios API client
│   ├── auth.ts            # Authentication service
│   └── storage.ts         # Local storage utilities
├── types/
│   ├── study.ts           # TypeScript interfaces
│   ├── user.ts            # User type definitions
│   └── api.ts             # API response types
└── utils/
    ├── constants.ts       # Application constants
    ├── formatters.ts      # Data formatting utilities
    └── validators.ts      # Form validation rules
```

5.2 Key Features

- **React Router**: Client-side routing with protected routes
- **Context API**: Global state management for authentication and app state
- **Axios Integration**: HTTP client with interceptors for authentication

- **shadcn/ui Components**: Modern, accessible UI component library
- **React Hook Form**: Form management with validation
- **Tailwind CSS**: Utility-first CSS framework

5.3 Authentication Flow

```
// Login flow
1. User submits credentials → AuthService.login()
2. API validates credentials → Returns JWT tokens
3. Tokens stored in secure storage → Update AuthContext
4. Protected routes accessible → API requests include Bearer token
5. Token refresh on expiration → Seamless user experience
```

6. Database Design (MySQL - RDS)

6.1 Core Tables Overview

```sql
-- Studies table (main entity)
CREATE TABLE studies (
    id INT PRIMARY KEY AUTO_INCREMENT,
    title VARCHAR(500) NOT NULL,
    description TEXT,
    study_type ENUM('impact', 'outcome', 'impact_outcome_story'),
    status ENUM('draft', 'published', 'archived') DEFAULT 'draft',
    created_by INT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (created_by) REFERENCES users(id)
);

-- Study-Indicator relationships (many-to-many)
CREATE TABLE studies_indicators (
    id INT PRIMARY KEY AUTO_INCREMENT,
    study_id INT NOT NULL,
    indicator_id INT NOT NULL,
    value DECIMAL(10,2),
    unit VARCHAR(100),
    FOREIGN KEY (study_id) REFERENCES studies(id) ON DELETE CASCADE,
    FOREIGN KEY (indicator_id) REFERENCES clarisa_indicators(id)
);

-- CLARISA reference data
CREATE TABLE clarisa_initiatives (
    id INT PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    description TEXT,
    active BOOLEAN DEFAULT TRUE
);

CREATE TABLE clarisa_centers (
    id INT PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    acronym VARCHAR(50),
    active BOOLEAN DEFAULT TRUE
);
```

```sql
CREATE TABLE clarisa_countries (
    id INT PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    iso_code VARCHAR(3),
    region VARCHAR(100)
);
```

6.2 Entity Relationships

- **Studies** (1:N) → **Study Contributors**
- **Studies** (N:M) → **Indicators** (via studies_indicators)
- **Studies** (N:M) → **Initiatives** (via study_initiatives)
- **Studies** (N:M) → **Countries** (via study_countries)
- **Users** (1:N) → **Studies** (created_by relationship)

6.3 Indexing Strategy

```sql
-- Performance optimization indexes
CREATE INDEX idx_studies_type_status ON studies(study_type, status);
CREATE INDEX idx_studies_created_by ON studies(created_by);
CREATE INDEX idx_studies_created_at ON studies(created_at);
CREATE INDEX idx_studies_indicators_study ON studies_indicators(study_id);
CREATE FULLTEXT INDEX idx_studies_search ON studies(title, description);
```

7. Infrastructure and Deployment (AWS SAM +

CloudFormation)

7.1 SAM Template Structure

```yaml
# template.yaml
AWSTemplateFormatVersion: '2010-09-09'
Transform: AWS::Serverless-2016-10-31

Parameters:
  Environment:
    Type: String
    Default: dev
    AllowedValues: [dev, staging, prod]

Globals:
  Function:
    Runtime: python3.11
    Timeout: 30
    Environment:
      Variables:
        ENVIRONMENT: !Ref Environment
        DB_HOST: !GetAtt Database.Endpoint.Address

Resources:
  # API Gateway
  ImpactCompendiumApi:
    Type: AWS::Serverless::Api
    Properties:
      StageName: !Ref Environment
      Cors:
        AllowMethods: "'*'"
        AllowHeaders: "'*'"
        AllowOrigin: "'*'"

  # Lambda Functions
  StudiesFunction:
    Type: AWS::Serverless::Function
    Properties:
      CodeUri: app/
      Handler: routers.studies.handler
      Events:
        StudiesApi:
```

```
          Type: Api
          Properties:
            RestApiId: !Ref ImpactCompendiumApi
            Path: /studies
            Method: ANY

  # RDS Database
  Database:
    Type: AWS::RDS::DBInstance
    Properties:
      DBInstanceClass: db.t3.micro
      Engine: mysql
      EngineVersion: '8.0'
      AllocatedStorage: 20
      StorageEncrypted: true
      VPCSecurityGroups:
        - !Ref DatabaseSecurityGroup

  # Cognito User Pool
  UserPool:
    Type: AWS::Cognito::UserPool
    Properties:
      UserPoolName: !Sub "${AWS::StackName}-users"
      AutoVerifiedAttributes:
        - email
```

7.2 Deployment Configuration

```
# samconfig.toml
[default.deploy.parameters]
stack_name = "impact-compendium"
s3_bucket = "impact-compendium-deployments"
s3_prefix = "sam-artifacts"
region = "us-east-1"
capabilities = "CAPABILITY_IAM"
parameter_overrides = "Environment=dev"
```

8. Security & Compliance

8.1 Authentication & Authorization

- **AWS Cognito**: Managed user authentication with MFA support
- **JWT Tokens**: Secure API access with configurable expiration
- **Role-based Access**: Admin, Researcher, Viewer permission levels
- **API Rate Limiting**: Prevent abuse with request throttling

8.2 Data Security

- **HTTPS Enforcement**: All communications encrypted in transit
- **Database Encryption**: RDS encryption at rest using AWS KMS
- **Secrets Management**: AWS Secrets Manager for database credentials
- **Parameter Store**: Secure configuration management via SSM

8.3 Compliance Measures

- **Audit Logging**: Comprehensive activity tracking
- **Data Retention**: Configurable data lifecycle policies
- **Access Monitoring**: CloudTrail integration for security auditing
- **Backup Strategy**: Automated RDS snapshots and point-in-time recovery

9. Scalability & Performance

9.1 Auto-scaling Configuration

- **Lambda Concurrency**: Automatic scaling based on request volume
- **RDS Scaling**: Read replicas for improved query performance
- **API Gateway Throttling**: Request rate limiting and burst handling
- **CloudFront CDN**: Static asset caching and global distribution

9.2 Performance Optimization

- **Database Indexing**: Optimized queries for search and filtering
- **Connection Pooling**: Efficient database connection management

- **Caching Strategy**: Redis/ElastiCache for frequently accessed data
- **Lazy Loading**: Frontend optimization with code splitting

9.3 Load Testing Strategy

- **Expected Throughput**: 1000 concurrent users, 10,000 studies
- **Performance Targets**: <2s page load, <500ms API response
- **Stress Testing**: Regular load testing with AWS Load Testing solution

10. Monitoring & Observability

10.1 Logging Strategy

```
# CloudWatch Logs integration
import logging
import json

logger = logging.getLogger(__name__)
logger.setLevel(logging.INFO)

def log_api_request(request, response, duration):
    logger.info(json.dumps({
        "event": "api_request",
        "method": request.method,
        "path": request.url.path,
        "status_code": response.status_code,
        "duration_ms": duration,
        "user_id": request.state.user_id
    }))
```

10.2 Metrics & Alarms

- **API Performance**: Response time, error rate, throughput
- **Database Metrics**: Connection count, query performance, storage usage
- **Lambda Metrics**: Invocation count, duration, error rate
- **Custom Business Metrics**: Study creation rate, user activity

10.3 Health Monitoring

```
# Health check endpoint
@app.get("/health")
async def health_check():
    return {
        "status": "healthy",
        "timestamp": datetime.utcnow().isoformat(),
        "version": "1.0.0",
        "database": await check_database_connection()
    }
```

11. CI/CD Workflow

11.1 Pipeline Stages

```yaml
# .github/workflows/deploy.yml
name: Deploy Impact Compendium

on:
  push:
    branches: [main, develop]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Run Tests
        run: |
          pip install -r requirements-dev.txt
          pytest tests/
          npm test

  build:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - name: Build Application
        run: |
          sam build
          npm run build

  deploy:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - name: Deploy to AWS
        run: |
          sam deploy --config-env ${{ github.ref_name }}
```

11.2 Rollback Strategy

- **CloudFormation Stack Rollback**: Automatic rollback on deployment failure
- **Database Migration Rollback**: Reversible Alembic migrations
- **Blue-Green Deployment**: Zero-downtime deployments with traffic shifting
- **Feature Flags**: Gradual feature rollout with instant rollback capability

12. Cost & Resource Optimization

12.1 Cost Structure

- **Lambda Functions**: Pay-per-request pricing model
- **RDS Instance**: On-demand pricing with reserved instance options
- **API Gateway**: Request-based pricing with caching
- **S3 Storage**: Tiered storage for backups and static assets

12.2 Optimization Strategies

- **RDS Auto-pause**: Automatic database shutdown during idle periods
- **Lambda Provisioned Concurrency**: Optimize for consistent performance
- **CloudWatch Log Retention**: Configurable log retention periods
- **S3 Lifecycle Policies**: Automatic archival of old backup data

12.3 Cost Monitoring

- **AWS Cost Explorer**: Regular cost analysis and optimization
- **Budget Alerts**: Automated notifications for cost thresholds
- **Resource Tagging**: Detailed cost allocation by environment and feature

13. Future Extensions

13.1 Integration Roadmap

- **PRMS/ROAR Integration**: Bidirectional data synchronization

- **PowerBI Connector**: Direct dashboard integration
- **External API Integration**: CLARISA data synchronization
- **Mobile Application**: React Native mobile client

13.2 Advanced Features

- **AI Text Mining**: Automated indicator extraction from study descriptions
- **Machine Learning**: Predictive analytics for impact forecasting
- **Workflow Engine**: Approval workflows for study publication
- **Advanced Analytics**: Statistical analysis and correlation tools

13.3 Technical Enhancements

- **GraphQL API**: Flexible data querying capabilities
- **Real-time Updates**: WebSocket integration for live collaboration
- **Microservices Architecture**: Service decomposition for scalability
- **Event-driven Architecture**: Asynchronous processing with EventBridge

14. Appendix

14.1 Acronyms

- **API**: Application Programming Interface
- **AWS**: Amazon Web Services
- **CGIAR**: Consultative Group for International Agricultural Research
- **CRUD**: Create, Read, Update, Delete
- **JWT**: JSON Web Token
- **ORM**: Object-Relational Mapping
- **RDS**: Relational Database Service
- **REST**: Representational State Transfer
- **SAM**: Serverless Application Model
- **SPA**: Single Page Application
- **VPC**: Virtual Private Cloud

14.2 References

- AWS SAM Documentation
- FastAPI Documentation
- React Documentation
- MySQL Documentation
- AWS RDS Documentation
- AWS Cognito Documentation

14.3 Contact Information

- **Project Lead**: IT Architecture Team
- **Technical Lead**: Backend Development Team
- **Frontend Lead**: UI/UX Development Team
- **DevOps Lead**: Infrastructure Team

Document Status: Draft
Next Review Date: November 21, 2025
Approval Required: Technical Architecture Board